Task 1: CoreML Export & Quantization Pipeline

1.Task Description

In this task, I aimed to optimize a Multi-modal Visual Entailment model (combining a ViT vision encoder and a BERT-based text encoder) for high-efficiency on-device inference. The goal was to bridge the gap between heavy research-grade PyTorch models and edge-ready deployment on macOS/iOS hardware.

My primary objectives were:

- Decoupling the monolithic architecture into modality-specific sub-graphs.
- Exporting and quantizing the Vision Encoder to INT8 format via ONNX.
- Integrating the quantized vision backbone with a full-precision fusion head using CoreML.
- Benchmarking the latency, throughput, and memory footprint on Apple Silicon (M-series) to prove the benefits of 4x model size reduction.

2. Procedure and Technical Implementation

2.1. Architectural Splitting (The Wrappers)

One cannot easily export a massive, dual-modality PyTorch model into a single hardware-optimized file. The script solves this by physically slicing the VisualEntailmentModel in half using two clever Python wrappers:

- VisionEncoderONNXWrapper: This class isolates *only* the ViT. It strips away the text and fusion logic so the script can export a pure image-in, vector-out pipeline.
- FusionHeadOnly: This isolates the SwiGLU classifier and the Cross-Attention fusion layers.

2.2. The Vision Pathway (ONNX & INT8 Quantization)

The visual encoder (ViT) is usually the heaviest part of the model.

- The Export: The export_vision_encoder_to_onnx() function uses torch.onnx.export with opset_version=14 (the standard for modern Transformers). It also applies dynamic_axes, which tells ONNX that the batch size can change (meaning it won't crash if I pass 4 images instead of 1).

- The Quantization: The quantize_onnx_int8() function uses ONNXRuntime's quantize_dynamic. It takes the massive 32-bit floating-point weights (FP32) and mathematically maps them into 8-bit integers (INT8). This slashes the file size and memory footprint of the ViT by ~75%.

2.3. The Logic Pathway (Apple CoreML)

Apple Silicon (M1/M2/M3 chips) has a dedicated piece of hardware called the Apple Neural Engine (ANE).

- The convert_fusion_head_to_coreml() function takes the FusionHeadOnly wrapper and uses torch.jit.trace to freeze the mathematical graph.
- It then passes that traced graph to coremltools.convert(), targeting macOS13. This compiles your SwiGLU logic into an .mlpackage that can run natively and ultra-efficiently on Apple hardware.

2.4. The Hybrid Inference Engine

Because the model was split into pieces, the script has to stitch them back together to actually make a prediction. This is handled by the hybrid_infer_batch() function. When an image and text pair comes in:

1. It passes the image to the INT8 ONNX model (using CPU or Edge hardware).
2. It passes the text to standard PyTorch BERT (which remains fast because text is lightweight).
3. It takes the two resulting vectors and passes them to the CoreML model to make the final "Entailment vs Neutral vs Contradiction" decision.

2.5. Rigorous Benchmarking & Verification

The script doesn't just guess that the new pipeline is better; it mathematically proves it.

- Latency & Throughput: The benchmark_pipeline() function loops through batch sizes (1, 4, 8). It uses time.perf_counter() to measure the exact milliseconds it takes to run, calculating the throughput (items per second).
- Memory Tracking: It uses the psutil library (process.memory_info().rss) to take a snapshot of your Mac's RAM usage before and after the inference, proving the memory reduction.
- Accuracy Retention: The verify_quantization_accuracy() function runs a "live race." It feeds 100 images to the heavy PyTorch model, and the *exact same 100 images* to the

Apple/ONNX pipeline. It compares the answers to guarantee that the INT8 rounding errors didn't destroy the model's logic.

3. Results and Evidence

I conducted extensive benchmarking on Mac M-series hardware to measure the impact of these optimizations.

3.1 Throughput, Latency and Memory Comparison

	engine	batch_size	latency_mean_ms	latency_p50_ms	latency_p90_ms	throughput_items_per_sec	rss_delta_mb
0	pytorch	1	21.264813	21.307354	21.491780	47.026043	11.656250
1	quantized_hybrid_coreml	1	31.746110	31.708562	32.507096	31.499922	4.265625
2	pytorch	4	45.417919	45.292125	46.102450	88.070967	8.250000
3	quantized_hybrid_coreml	4	90.688475	90.371354	92.733234	44.107038	48.421875
4	pytorch	8	84.497773	84.522083	84.695308	94.677051	7.140625
5	quantized_hybrid_coreml	8	184.667804	176.664521	206.800092	43.321033	68.640625

Figure 1: Snapshot of comparison of latency(mean,p50,p90), throughput, memory requirements of PyTorch model vs Quantized hybrid model

3.2 Accuracy Comparison

```
--- STEP 5: QUANTIZATION ACCURACY VERIFICATION ---
Evaluating 100 samples to verify accuracy retention...

--- ACCURACY RESULTS ---
Original PyTorch Accuracy : 59.0%
Quantized Apple Accuracy  : 58.0%
Model Agreement (Retention): 96.0%
SUCCESS: Quantized model maintained >95% identical predictions.
Saved summary: task1_artifacts/task1_summary.json
```

Figure 2: Snapshot of the accuracy of both the models on 100 test samples

4. Results Analysis & Hardware Diagnostics

4.1 Memory Footprint Validation (Single-Batch)

Testing peak RAM usage (RSS delta) at a batch size of 1 proved highly successful:

- PyTorch Baseline: +11.66 MB RSS Spike
- Quantized CoreML Hybrid: +4.27 MB RSS Spike

Compressing the massive FP32 Vision Transformer weights to 8-bit integers achieved a nearly 3x memory reduction. This ensures the optimized model runs safely on low-RAM mobile devices without triggering Out-Of-Memory (OOM) OS faults.

4.2 Latency Benchmarks and the "I/O Tax"

Instead of a 3-5x speedup, a latency inversion occurred at batch size 1: the PyTorch baseline took 21.2ms ($p_{50}: 21.3ms$), while the hybrid pipeline took 31.7ms ($p_{50}: 31.7ms$). This is due to the Python Serialization "I/O Tax":

- PyTorch MPS Advantage: The baseline natively uses Apple's Metal Performance Shaders (MPS), executing entirely in Unified Memory with zero cross-framework data movement.
- Hybrid Overhead: The optimized pipeline forces serial data transfers across execution providers: ONNXRuntime (CPU fallback), PyTorch (GPU), and CoreML (Apple Neural Engine). The physical copying time in Python heavily outweighs the computational gains of INT8 operations.

4.3 Batched Memory Scaling Anomalies

At Batch Size 8, PyTorch memory spiked by only 7.1 MB, whereas the Hybrid pipeline spiked by 68.6 MB. PyTorch MPS efficiently allocates unified memory for batched tensors. In contrast, bridging ONNX, PyTorch, and CoreML in Python requires dynamically allocating intermediate translation buffers that scale linearly, eventually overriding the static INT8 memory savings.

4.4 Architectural Decoupling: The Hybrid Wrapper Justification

I intentionally decoupled the architecture rather than compiling a single .mlpackage. ONNXRuntime dynamic quantization introduces DynamicQuantizeLinear nodes that Apple's coremltools compiler struggles to ingest, causing translation failures. To satisfy the project's ONNXRuntime requirement while leveraging CoreML, my Python wrapper safely bridged these incompatible graphs at the cost of I/O latency.

5. Path to Production (Conclusion)

This experiment validated the mathematical accuracy (>95%) and physical footprint compression of the INT8/CoreML export. However, Python is inadequate for true edge latency. To fully realize the 3-5x speedups and eliminate memory overhead, the entire FP32 graph should

be unified, natively quantized via CoreML APIs (bypassing ONNXRuntime), and deployed in a native Swift/iOS app to execute directly on the Apple Neural Engine.

File Structure

```
Visual_Entailment_Project/
│
├── task1_export_pipeline.py            # The main script we just ran
├── final_sota_visual_entailment3.pth   # Your saved PyTorch weights
├── cleaned_snli_ve_test.csv            # The dataset used for benchmarking
│
└── task1_artifacts/                    # Auto-generated by the script
    │
    ├── vision_encoder_opset14.onnx     # Step 1: Heavy FP32 Vision Model (~340MB)
    ├── vision_encoder_int8.onnx        # Step 2: Compressed INT8 Vision Model (~85MB)
    │
    ├── fusion_head_fp16.mlpackage/     # Step 3: Apple CoreML Logic Engine
    │   ├── Data/
    │   └── Manifest.json               # (CoreML saves as a folder structure)
    │
    ├── benchmark_results.csv           # Step 4: The Latency/Memory metrics
    └── task1_summary.json              # Step 5: Reproducibility metadata
```